Objectives

The Problem to Solve

Luxury vehicle owners in urban environments face challenges in securing their cars due to limited and often inadequate parking security. Areas like Manhattan, where high-value vehicles are common, require advanced solutions to prevent unauthorized access and ensure safe parking. Existing systems lack comprehensive access control and collision prevention, increasing the risk of security breaches or vehicle damage.

The Solution

To address this issue, we propose an automated underground parking system that enhances security, safety, and user convenience. Key components include an LDR sensor for vehicle detection, a matrix keypad for secure access control, LED indicators to signal safe entry, and an ultrasonic distance sensor to prevent collisions. A buzzer alerts drivers when they are too close to the wall, and an exit button allows a seamless departure. For added security, users can reset their access code from inside the parking spot.

General Goals

- Provide a secure and convenient parking experience for luxury vehicle owners.
- Ensure the system is reliable and responsive to user input and car detection.
- Minimize the risk of vehicle damage through collision prevention.
- Enhance security through effective access control and alert systems.
- Offer an intuitive interface and ease of use for drivers.

Performance Goals

- Access Control: The system should process code input within 2 seconds and open the gate within 3 seconds upon successful authentication.
- Collision Prevention: The ultrasonic sensor should detect obstacles within 5 cm and trigger the buzzer in less than 500ms.
- Security Measures: The system will lock out after three incorrect attempts and activate an alarm.

Functions to be Achieved

The system follows a structured sequence:

1. The vehicle is detected at the entrance.
2. The user enters a security code.
3. If verified, the gate opens, and the system signals safe entry with LEDs.
4. The ultrasonic sensor prevents collisions while parking.
5. An exit button allows the vehicle to leave.
6. If an incorrect code is entered three times, the system activates an alarm.

Design Specifications and Expected Behavior

In this section, we outline the design specifications for the system and describe the expected behaviors in various operational scenarios. The table below summarizes the detailed specifications, expected behaviors, and how the system should respond in different cases.

Specification	**Description**	**Expected Behavior**
Correct Code Entered	The user enters the correct code within 3 attempts.	The gate opens, and the system displays “Welcome!” before closing the gate after 5 seconds.

Car Detected with Correct Code	A car is detected at the entrance, and the correct code is entered within the allowed attempts.	The system grants access, opens the gate, and displays a welcome message.
Multiple Incorrect Attempts (3)	The user enters 3 incorrect codes.	After 3 failed attempts, the system triggers a security issue, activates the alarm, and displays “3 WRONG ATTEMPTS” and “ALARM ACTIVATED!”.
Successful Gate Operation	After entering the correct code, the gate opens and closes as expected.	The gate opens for 5 seconds and then closes. While opening or closing, redLed should be on. While gate is open, greenLed should be on
Display Updates Correctly	The display shows the correct information during each attempt.	The system displays “3 10sec Attempts”, the attempt number, and correct status messages (“Welcome!”, “ALARM ACTIVATED!”, etc.).
Siren Activated After 3 Failures	After 3 incorrect attempts, the siren activates.	The siren should be turned on, and the system should display “3 WRONG ATTEMPTS” and “ALARM ACTIVATED!”.
Security Issue Cleared After Success	Once the correct code is entered, the system clears the security issue.	The securityIssue flag is reset to false, and the system proceeds as normal.
Car Detected and System Activated	The system detects a car at the entrance and activates the entrance subsystem.	The entrance system is activated and ready to receive code input.
Attempt Number Displayed Correctly	Each attempt number is displayed correctly on the screen.	The display shows the current attempt number in the format “Attempt X:”.

Table 1: Function Table

Table 1 summarizes the expected behaviors and outcomes of the system in various scenarios. Each row corresponds to a different aspect of system operation, detailing the condition (e.g., entering a correct code) and the expected response from the system (e.g., opening the gate and

displaying a welcome message). This table ensures clarity in understanding how the system should behave in real-world usage.

Design

Design Overview

The Automated Luxury Underground Parking System integrates hardware components and embedded software to ensure secure, convenient, and automated vehicle parking. The system combines sensor-based detection, access control, collision prevention, and automated gate operation, all managed by an STM32 Nucleo microcontroller.

The hardware components include:

- LDR sensor for vehicle detection
- Matrix keypad for secure code entry
- Ultrasonic sensor for collision prevention
- Positional servo motor to control garage door movement
- LED indicators for status signaling
- Buzzers for security and proximity alerts
- LCD display for real-time user feedback
- Exit and reset buttons for additional user functionality

The software is designed to:

- Process user input from the keypad and sensors
- Control hardware components based on authentication and safety conditions
- Trigger security alerts upon multiple incorrect attempts

- Prevent vehicle collisions through real-time distance monitoring
- Provide user-friendly interaction via LED signals and display messages

This integrated approach ensures a reliable, efficient, and user-friendly parking experience while enhancing security and vehicle safety.

Major Design Steps

The design process followed a structured approach:

1. Concept Development
    - Created an initial sketch outlining system components and layout.
    - Defined functional requirements such as access control, collision detection, and exit mechanisms.
2. Hardware Implementation
    - Selected and tested components, including sensors, actuators, and power sources.
    - Designed a block diagram to visualize input/output relationships.
    - Developed a detailed schematic using EasyEDA to finalize wiring and connections.
3. Software Development
    - Defined software modules for user authentication, sensor data processing, and actuator control.
    - Implemented real-time event handling using an embedded C program on the STM32 Nucleo board.
    - Developed a state-machine approach to ensure efficient code execution.
4. System Integration & Testing

- Assembled the hardware prototype, ensuring proper connections and power distribution.
- Tested each component individually, then conducted full system testing for functionality validation.
- Debugged issues, optimized response times, and ensured error handling mechanisms were effective.

Subsections to Follow

The following subsections provide a detailed breakdown of the system:

- System Overview: Explains the physical structure and overall function of the parking system.
- Hardware Design: Describes the key components, their electrical connections, and power requirements.
- Software Design: Details the software architecture, logic flow, and modular implementation.

This structured approach ensures a clear understanding of how the hardware and software work together to achieve an efficient and secure automated parking system.

System Overview

The system is built as a scaled-down model of an underground parking facility, using a styrofoam enclosure to simulate a real-world environment. Inside this model, various sensors and actuators interact to regulate vehicle entry, prevent collisions, and ensure secure access control.

Figure 1 presents an initial design drawing of the parking system. The front and inside views depict the key structural components, including the vehicle detection zone, garage door mechanism, sensor placements, and keypad interface.

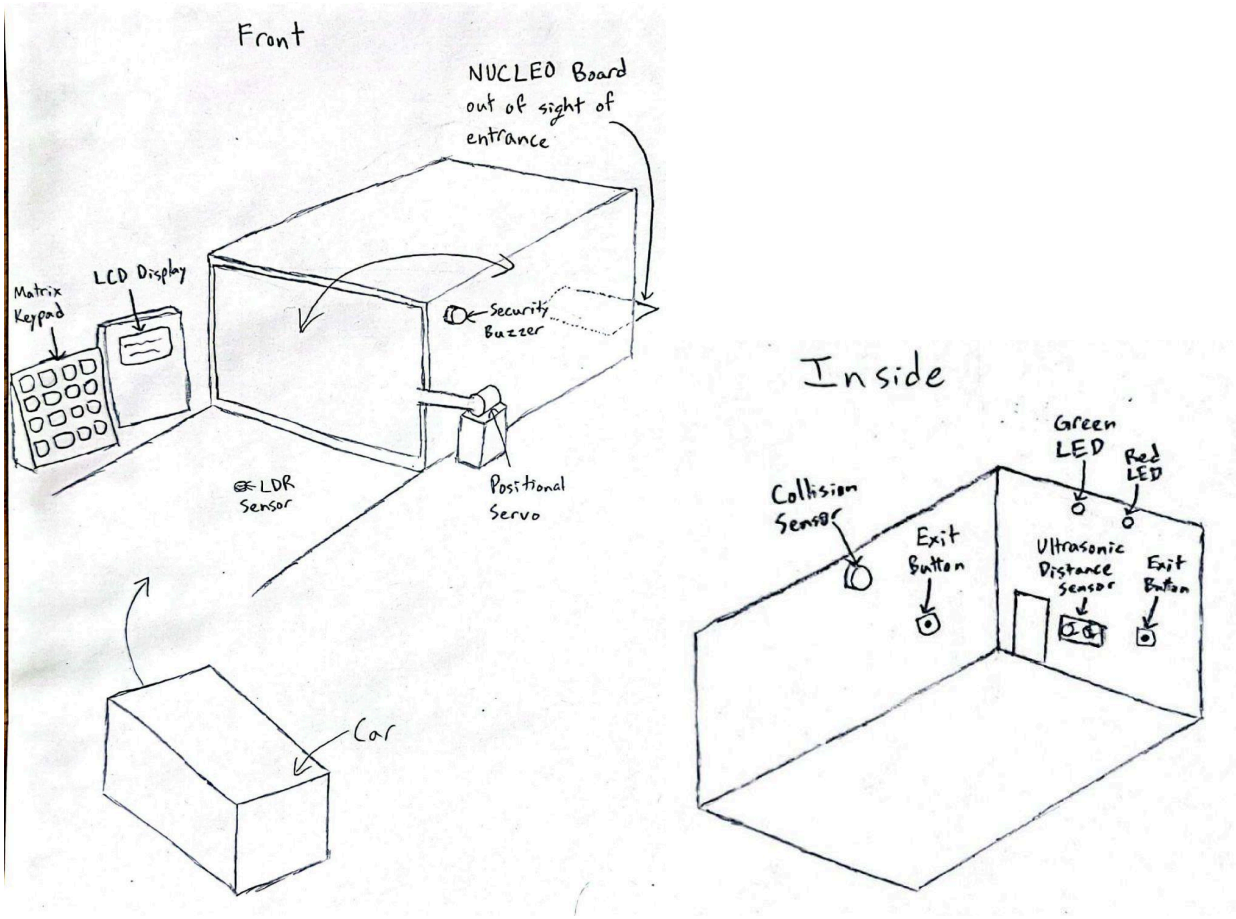

Figure 1: Initial Design Drawing – Front and Inside View

As seen in Figure 1, the parking garage is represented by a large styrofoam box, with a smaller box simulating the car. The entry system is triggered when the car moves over a light-dependent resistor (LDR) sensor, which is constantly exposed to light. When the car covers the sensor, it detects a drop in light level, activating the authentication process through a matrix keypad. The

driver must enter a valid 6-digit access code to gain entry.

An LCD display located next to the keypad provides real-time feedback. When the LDR detects a car, the display prompts the driver to input their code and shows which attempt (1, 2, or 3) they are currently on. If the user enters an incorrect code three times, a buzzer mounted externally on the side of the garage is activated, and the display will show "3 WRONG ATTEMPTS" and "ALARM ACTIVATED!" to indicate a security breach.

If authentication is successful, a positional servo motor lifts the garage door. While the door is in motion, a red LED turns on, signaling that entry is not yet safe. When the gate is fully open, a green LED turns on, indicating that the driver may proceed. After 5 seconds, the gate automatically closes, and the red LED turns on again while the door is in motion.

Inside the garage, an ultrasonic distance sensor continuously monitors the car's position. If the car moves within 4.5 cm of the wall, an internal collision alarm is triggered to prevent accidents.

To exit, the driver can press an exit button located on the side of the wall, which will open the garage door for departure. Additionally, a reset button inside the garage allows the user to change their access code, a function we demonstrated using the serial monitor.

All components are controlled by the STM32 NUCLEO board, which is located on the exterior of the structure.

Hardware

The hardware design of our automated luxury underground parking system integrates various sensors, actuators, and feedback components, all controlled by an STM32 NUCLEO board. This

section describes the major hardware components, their interfaces, and how they interact to implement system functions.

To systematically organize the hardware layout, we first developed a top-level block diagram, which maps out the inputs and outputs and their respective signal types.

System Inputs and Outputs

Figure 2 illustrates the primary hardware components and their respective connections to the Nucleo board. The system utilizes a mix of digital, analog, and PWM signals to process user inputs and control mechanical actions.

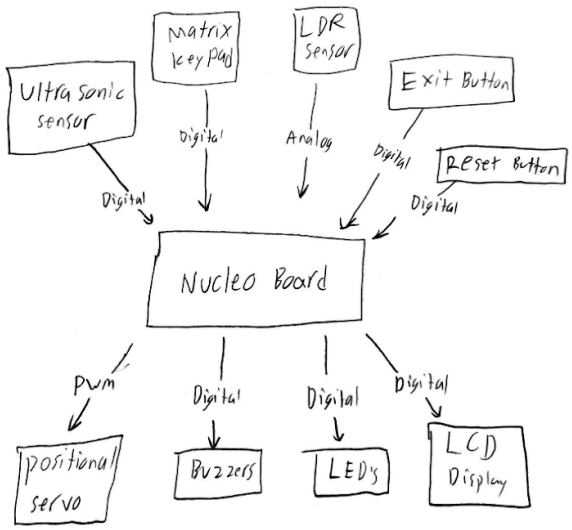

Figure 2: Input/Output Block Diagram

The inputs to the system include:

- Keypad – Digital input for user authentication.
- LDR Sensor – Analog input (3.3V) for detecting vehicle presence.
- Ultrasonic Sensor – Digital input (5V) for distance measurement and collision

prevention.

- Exit Button – Digital input (3.3V) for opening the gate upon exit.
- Reset Button – Digital input (3.3V) for resetting user access.

The outputs of the system include:

- Buzzer – Digital output (5V) to indicate security alerts.
- Positional Servo (Gate Control) – PWM output (5V) to open and close the gate.
- LED Indicators – Digital output (3.3V) to signal entry status.
- LCD Display – Digital output (5V) to provide system messages.

This block diagram provides a clear overview of how data flows between the user, the environment, and the Nucleo board, ensuring seamless system operation.

Interfaces and Communication Protocols

While most sensors and actuators use standard digital or analog signals, some require specific communication protocols for proper operation:

- Keypad: Uses a row-column scanning technique to detect keypresses efficiently.
- Ultrasonic Sensor (HC-SR04): Operates by sending a trigger pulse and measuring the time delay of the returning echo pulse to determine distance.
- LCD Display: Utilizes a parallel data interface to quickly display system messages.
- Positional Servo: Controlled using a PWM (Pulse Width Modulation) signal, where the duty cycle determines its angular position.

By managing these interfaces effectively, the system ensures reliable and responsive operation.

Schematic and Component Connections

After defining the block diagram, we created a detailed schematic in EasyEDA, mapping out all hardware connections. Figure 3 below presents the circuit schematic, showing how each component is wired to the STM32 NUCLEO board and power sources.

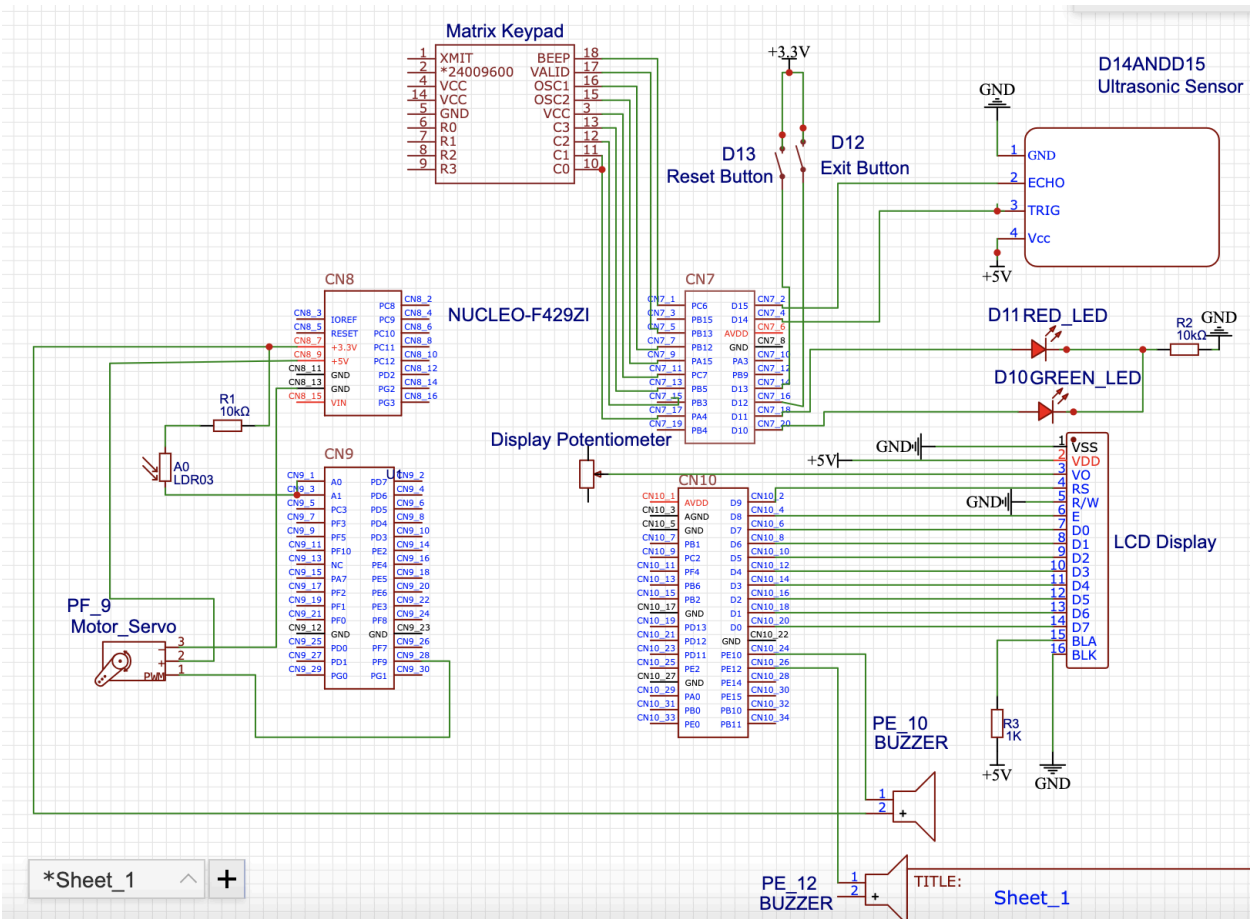

Figure 3: Hardware System Schematic

In this schematic, all power connections, pin assignments, and external components are clearly outlined. The system components are powered through either 3.3V or 5V supplied by the Nucleo board and MB102 power module.

Key wiring considerations:

- Pin Assignments: Each GPIO pin is mapped for reliable signal transmission.
- Power Distribution: The system is designed to prevent overloading the Nucleo board's power supply.
- Common Grounding: All components share a common ground to avoid electrical instability.

This schematic serves as a blueprint for assembling and troubleshooting the system, ensuring proper integration of all hardware elements.

Software

The parking garage system is a modular embedded system running on an STM32 Nucleo board. The software is responsible for vehicle access control, security enforcement, collision prevention, and system monitoring via serial communication.

Software Structure

The system follows a modular architecture, where each component has a dedicated role. The parking_system.cpp file serves as the central control unit, managing initialization and real-time updates of all subsystems.

Module Diagram

The following diagram illustrates the software architecture and interaction between different modules. Each module has a specific role, ensuring efficient and organized execution of tasks. The modular approach enables scalability, maintainability, and enhanced debugging capabilities.

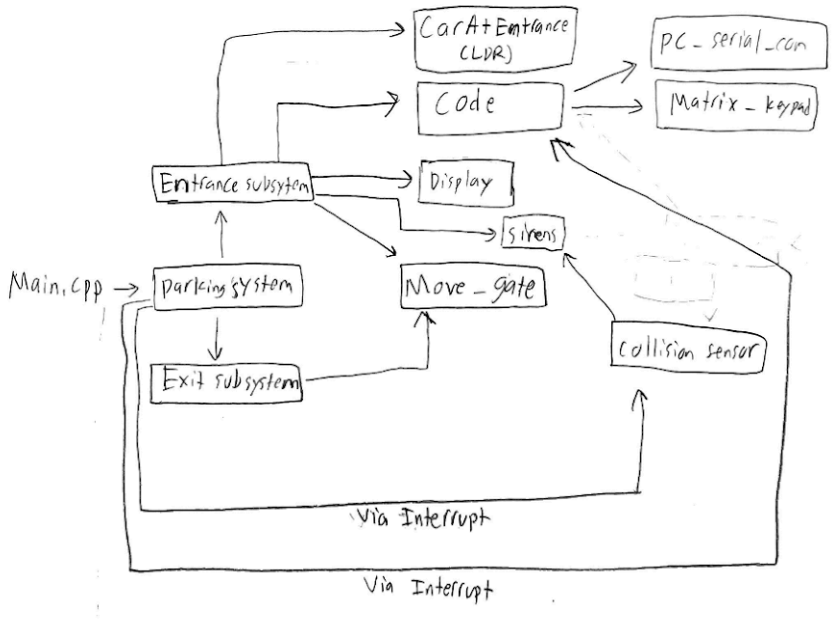

Figure 4: Software Module Diagram

This diagram provides a visual representation of the system's functional breakdown, showing the interaction between modules such as gate control, collision detection, security enforcement, and user interface management.

Module Breakdown

1. Main Control (main.cpp)
    - Calls parkingSystemInit() to initialize all subsystems.
    - Runs a continuous loop to update the system periodically via parkingSystemUpdate().
2. Parking System Module (parking_system.cpp)
    - Handles system-wide initialization and updates all subsystems every 10ms.
    - Calls parkingSystemInit() to initialize:

        - Security system (sirensInit()).
        - User interface (displayInit(), matrixKeypadInit()).
        - Gate control (moveGateInit()).
        - Entry and exit handling (carAtEntranceInit(), entranceSubsystemInit(), exitSubsystemInit()).
        - Collision detection (distanceSensorInit(), collisionSensorInterrupt()).
        - Code reset mechanism (codeResetButtonInterrupt()).
        - Serial communication (pcSerialComInit()).
    - Calls parkingSystemUpdate() to ensure real-time updates.
3. Entrance Subsystem (entrance_subsystem.cpp)
    - Detects a vehicle at the entrance.
    - Prompts the driver to enter an access code on the keypad.
    - Displays attempt count and allows 3 attempts.
    - If correct code entered:
        - Displays "Welcome!".
        - Opens the gate for 5 seconds, then closes it automatically.
    - If 3 incorrect attempts occur:
        - Activates external siren (externalSirenStateWrite(ON)).
        - Displays "3 WRONG ATTEMPTS" and "ALARM ACTIVATED!".
4. Exit Subsystem (exit_subsystem.cpp)
    - Detects when a user presses the exit button.
    - Opens the garage door for 5 seconds.

    - Closes the door automatically after delay.

5. Gate Control (move_gate.cpp)
    - Uses a servo motor to control the garage gate.
    - LED indicators:
        - Red LED signals the gate is moving.
        - Green LED signals the gate is open.
    - Smooth gate movement using PWM control:
        - openGate() gradually opens the gate.
        - closeGate() gradually closes the gate.
6. Security & Alarm System (sirens.h and sirens.cpp)
    - Manages internal and external sirens for security enforcement.
    - Internal siren is triggered when a car is too close to the wall.
    - External siren is activated when 3 failed code attempts occur.
    - sirensInit() initializes the siren state to OFF.
    - sirensUpdate() ensures sirens turn ON/OFF based on system conditions.
7. Collision Detection (collision_sensor.h and collision_sensor.cpp)
    - Uses an ultrasonic sensor to detect if a car is too close to a wall.
    - Triggers the internal siren if a collision is imminent.
    - distanceSensorInit() initializes distance sensor readings.
    - collisionSensorUpdate() checks distance and activates the siren if needed.
8. Interrupt Handling (interrupt.h and interrupt.cpp)
    - Handles hardware interrupts for:

        - Code reset button (codeResetButtonInterrupt()).
        - Collision detection updates (collisionSensorInterrupt()).
    - codeResetButtonInterrupt() initializes an interrupt for resetting the access code when the button is pressed.
    - collisionSensorInterrupt() uses a Ticker to periodically check for collisions every 10ms.

9. Matrix Keypad (matrix_keypad.h and matrix_keypad.cpp)
    - Handles 4x4 keypad input for access code entry.
    - Implements debounce filtering to prevent false key presses.
    - matrixKeypadUpdate() scans keys and returns the released key value.
10. Car at Entrance Detection (car_at_entrance.h and car_at_entrance.cpp)
    - Uses a light sensor to detect vehicle presence at the entrance.
    - Implements rolling average filtering to smooth light sensor readings.
    - carAtEntranceInit() initializes the sensor array and updates readings.
    - carIsDetected() determines whether a car is present based on filtered readings.
11. User Interface Components

- Display (display.h and display.cpp)
    - Updates messages on the LCD screen based on system status.
- Code Entry & Reset (code.h and code.cpp)
    - Handles code verification and resetting access codes.

12. PC Serial Communication (pc_serial_com.h and pc_serial_com.cpp)
    - Enables USB-based serial communication for debugging and monitoring.

- pcSerialComStringWrite() allows the system to send messages over the serial port.
- pcSerialComCharRead() enables reading single-character input from the serial interface.

Software Flow & Logic

The system runs on a real-time update cycle of 10ms, ensuring continuous operation. This modular approach ensures efficient, scalable, and maintainable software design.

Results and Discussion

Observed Behavior

To validate the performance of our automated parking system, a series of structured tests were conducted under different conditions. Each component was initially assessed independently before being integrated into the overall system to verify responsiveness and reliability. The table below outlines the test cases, expected behavior, actual results, and overall success rate.

Test	Expected Outcome	Observed Outcome	Result
LDR detects vehicle presence	Sensor reliably detects vehicle presence	Vehicle detection was 100% accurate	PASS
Keypad authentication (valid code)	Gate opens immediately and remains open for 5 seconds	Gate functioned correctly, staying open for 5 seconds before closing	PASS
Keypad authentication (invalid code)	Access denied, buzzer sounds after three failed attempts	System denied access, activated buzzer, and displayed "ALARM ACTIVATED" on LCD	PASS
Ultrasonic sensor collision alert	Buzzer activates at a 4.5cm threshold	Buzzer consistently activated at 4.5cm	PASS

Gate safety mechanism (obstruction detected)	Gate should not close if an obstruction is present	System failed to prevent closure, indicated need for refinement	FAIL
Exit button functionality	Pressing button should trigger gate opening	Gate opened successfully	PASS
Reset button (code change)	Allows user to change code via serial monitor	Code modification was successfully executed	PASS

Table 2: Test Results and System Performance

Based on these findings, all expected functionalities were met except for the gate safety mechanism, which requires an additional interrupt on the distance sensor to ensure effective obstruction detection.

Code Maintainability

The system's software architecture was developed with maintainability as a priority to facilitate troubleshooting, scalability, and future enhancements. The code is structured modularly, with distinct functions dedicated to sensor input processing, authentication, gate operation, and collision prevention. This design approach ensures that modifications to one feature do not interfere with the rest of the system.

To enhance readability and ease of modification, clear comments and documentation are incorporated throughout the code. Additionally, instead of embedding fixed values, configurable constants are used for key parameters such as sensor thresholds, timing delays, and authentication constraints. This allows for quick and efficient adjustments without the need for extensive code revisions, promoting a flexible and adaptable system.

Debugging

Several issues emerged during development, necessitating debugging efforts to identify and resolve system failures. One recurring problem involved the code breaking after integrating specific modules. While merging individual components into parking_system.cpp, certain modules unexpectedly caused malfunctions in others. For instance, declaring a PwmOut servo pin led to disruptions in LED, buzzer, and LCD operation. Commenting out the declaration restored functionality, yet the exact cause remained elusive even after reviewing the code line by line and consulting the professor.

To address this, we rebuilt the project based on a prior stable version (Project 3) that successfully integrated a PwmOut servo module alongside an LCD display. By systematically incorporating existing functionalities and filling in missing logic, we managed to restore the system's operation. This process had to be repeated multiple times as similar failures resurfaced, necessitating the creation of new repositories to reestablish stability.

The table below outlines specific troubleshooting scenarios and their respective solutions:

Fail Case	Description	Solution
Car Not Detected	System fails to recognize a vehicle at the entrance	Verify sensor alignment and confirm car is within detection range
Gate Fails to Open	Gate remains closed after correct code entry	Check motor functionality and ensure openGate() executes as expected
Display Malfunction	Incorrect or missing messages on LCD	Inspect display wiring and verify displayCharPositionWrite and displayStringWrite calls

Code Validation Error	isCodeCorrect() does not verify input correctly	Debug isCodeCorrect() logic for miscalculations
Timing Discrepancy	10-second attempt limit is inconsistent	Adjust delay functions and ensure timing conditions execute correctly
Siren Failure	External siren does not activate after multiple failed attempts	Check siren control logic (externalSirenStateWrite(ON)) and function sirensUpdate()
Sensor Update Issue	Sensors do not refresh properly after three failed attempts	Ensure sensorUpdate() executes as required and updates sensor data correctly

Table 3: Common Debugging Cases and Solutions

These debugging cases highlight the challenges encountered during development and the steps taken to resolve them. By systematically analyzing each failure and implementing corrective actions, we improved the system's overall stability and reliability. The experience reinforced the importance of incremental testing, code modularity, and maintaining backup versions to quickly recover from unexpected failures.

Improvements

If additional time were available, several enhancements could be implemented to improve system performance, security, and safety.

1. RFID-Based Authentication: One major enhancement would be the integration of an RFID authentication system alongside the keypad. Initially planned but not implemented due to time constraints, RFID-based access control would enhance security by requiring both a valid access code and an authorized RFID card. This dual-factor authentication method would significantly mitigate unauthorized access attempts.

2. Obstruction Detection for Gate Safety: Another critical improvement would involve implementing a mechanism to detect objects obstructing the gate during closure. In the current design, the servo motor lowers the garage door without verifying whether a vehicle remains in the entryway. A practical solution would be to incorporate an LDR sensor or an additional ultrasonic sensor to detect obstructions. If an object is detected, the system could halt or delay the closing process, thereby preventing damage to vehicles or potential injury to individuals. While an attempt was made to integrate this feature following the Tuesday presentation, time constraints prevented its full implementation.

By integrating these improvements, the automated parking system could become more secure, reliable, and aligned with real-world applications.